

PAC-MAN GAME

A fully playable Pac-Man built from scratch in Java—no database, just Swing and pure OOP.

● Week 4 Complete

Java JDK 21

VS Code

Swing / JPanel

60 FPS Game Loop

WHAT WAS BUILT

ScoreManager Class	Dot Eating & HUD
✓ ScoreManager.java — new scoring class ✓ addDot() adds 10 points per dot eaten ✓ addPowerPellet() adds 50 points ✓ loseLife() and isGameOver() logic ✓ countDots() in Maze for win detection	✓ eatDot() in PacMan removes tile from maze ✓ HUD shows live score and life circles ✓ Score updates every frame dots are eaten ✓ YOU WIN overlay when all dots cleared ✓ GAME OVER overlay when lives reach zero

KEY TECHNIQUES

eatDot() — Tile Removal on Collision

Each frame, after PacMan moves, eatDot() checks the tile at PacMan's current row/col. If it is a dot (2) or power pellet (3), it calls maze.setTile(row, col, 0) to erase it from the grid and returns the tile type. GamePanel reads the return value and tells ScoreManager to add the correct points.

ScoreManager — Separation of Concerns

Score and lives logic lives entirely in ScoreManager.java, separate from rendering and movement. GamePanel calls addDot(), addPowerPellet(), and loseLife() as events happen. This keeps each class focused on one job and makes it easy to extend scoring rules later.

Win Condition via countDots()

Maze.countDots() loops the full grid each frame counting tiles with value 2 or 3. When it returns 0, every dot and power pellet has been eaten and GamePanel sets won = true, freezing movement and showing the YOU WIN overlay. This is simple and reliable — no separate counter to keep in sync.

SOURCE CODE

ScoreManager.java

java

```
public class ScoreManager {  
  
    private int score;  
    private int lives;  
  
    public ScoreManager() {  
        this.score = 0;  
    }  
}
```

```

        this.lives = 3;
    }

    public void addDot() { score += 10; } public
    void addPowerPellet() { score += 50; } public
    void loseLife() { lives--; }

    public int getScore() { return score; } public int
    getLives() { return lives; } public boolean
    isGameOver() { return lives <= 0; }
}

```

Maze.java

java

```

import java.awt.*;

public class Maze {
    public static final int TILE_SIZE = 24;
    public static final int ROWS = 21;
    public static final int COLS = 19;

    private int[][] grid = {
        {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1},
        {1,2,2,2,2,2,2,2,2,1,2,2,2,2,2,2,2,2,1},
        {1,2,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,2,1},
        {1,3,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,3,1},
        {1,2,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,2,1},
        {1,2,2,2,2,2,2,2,2,1,2,2,2,2,2,2,2,2,1},
        {1,2,1,1,1,1,2,1,1,1,1,1,2,1,1,1,1,2,1},
        {1,2,1,1,1,1,2,1,1,1,1,1,2,1,1,1,1,2,1},
        {1,2,2,2,2,2,2,2,1,1,0,1,2,2,2,2,2,2,1},
        {1,1,1,1,1,2,1,1,1,1,1,1,1,2,1,1,1,1,1},
        {0,0,0,0,1,2,1,0,0,0,0,0,1,2,1,0,0,0,0},
        {1,1,1,1,1,2,1,1,1,1,1,1,1,2,1,1,1,1,1},
        {1,2,2,2,2,2,2,2,1,1,1,1,1,2,2,2,2,2,1},
        {1,2,1,1,1,1,2,1,1,1,1,1,2,1,1,1,1,2,1},
        {1,2,1,1,1,1,2,1,1,1,1,1,2,1,1,1,1,2,1},
        {1,2,2,2,2,2,2,2,2,1,2,2,2,2,2,2,2,2,1},
        {1,2,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,2,1},
        {1,3,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,3,1},
        {1,2,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,2,1},
        {1,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,1},
        {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1}
    };

    public int getTile(int row, int col) {
        if (row<0||row>=ROWS||col<0|| col >= COLS) return 1;
        return grid[row][col];
    }

    public void setTile(int row, int col, int value) { grid[row][col] = value; }
    public boolean isWall(int row, int col) { return getTile(row, col) == 1; }

    public int countDots() {
        int count = 0;
        for (int r = 0; r < ROWS; r++)
            for (int c = 0; c < COLS; c++)
                if (grid[r][c]==2||grid[r][c] == 3) count++;
    }
}

```

```

        return count;
    }

    public void draw(Graphics g) {
        Graphics2D g2 = (Graphics2D) g;
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
        for (int row=0; row< ROWS; row++) {
            for (int col=0; col<COLS; col++) {
                int tile = grid[row][col];
                int x = col * TILE_SIZE;
                int y = row * TILE_SIZE;
                if (tile == 1) {
                    g2.setColor(new Color(20, 20, 90));
                    g2.fillRoundRect(x+1, y+1, TILE_SIZE-2, TILE_SIZE-2, 6, 6);
                    g2.setColor(new Color(33, 99, 255));
                    g2.setStroke(new BasicStroke(2));
                    g2.drawRoundRect(x+1, y+1, TILE_SIZE-2, TILE_SIZE-2, 6, 6);
                } else if (tile == 2) {
                    g2.setColor(new Color(255, 204, 153));
                    g2.fillOval(x+TILE_SIZE/2 - 2, y + TILE_SIZE/2 - 2, 4, 4);
                } else if (tile == 3) {
                    g2.setColor(new Color(255, 204, 153));
                    g2.fillOval(x+TILE_SIZE/2 - 5, y + TILE_SIZE/2 - 5, 10, 10);
                }
            }
        }
    }
}

```

PacMan.java

java

```

import java.awt.*;
import java.awt.image.BufferedImage;
import javax.imageio.ImageIO;
import java.io.File;

public class PacMan {
    private int row, col;
    private double x, y;
    private int dx, dy;
    private int nextDx, nextDy;
    private double speed = 2.0;
    private BufferedImage sprite;
    private Maze maze;

    public PacMan(Mazemaze, int startRow, int startCol) {
        this.maze = maze;
        this.row = startRow; this.col = startCol;
        this.x = col * Maze.TILE_SIZE;
        this.y = row * Maze.TILE_SIZE;
        this.dx = 0; this.dy = 0;
        this.nextDx = 0; this.nextDy = 0;

        try {
            sprite = ImageIO.read(new File("assets/sprites/PacMan.png"));
        } catch (Exception e) {
            System.out.println("Could not load PacMan.png: " + e.getMessage());
        }
    }
}

```

```

    }

    public void setDirection(int newDx, int newDy) { nextDx = newDx; nextDy = newDy; }

    // Returns tile type eaten: 2=dot, 3=power pellet, 0=nothing
    public int eatDot() {
        int tile = maze.getTile(row, col);
        if (tile == 2 || tile == 3) {
            maze.setTile(row, col, 0);
            return tile;
        }
        return 0;
    }

    public void update() {
        if (canMove(nextDx, nextDy)) { dx = nextDx; dy = nextDy; }
        if (canMove(dx, dy)) {
            x += dx * speed;
            y += dy * speed;
        } else {
            x = col * Maze.TILE_SIZE;
            y = row * Maze.TILE_SIZE;
            dx = 0; dy = 0;
        }
        row = (int) Math.round(y / Maze.TILE_SIZE);
        col = (int) Math.round(x / Maze.TILE_SIZE);
        if (x < -Maze.TILE_SIZE) x = Maze.COLS * Maze.TILE_SIZE;
        if (x > Maze.COLS * Maze.TILE_SIZE) x = -Maze.TILE_SIZE;
    }

    private boolean canMove(int testDx, int testDy) {
        if (testDx == 0 && testDy == 0) return false;
        double nextX = x + testDx * speed;
        double nextY = y + testDy * speed;
        int checkRow = (int) Math.round(nextY / Maze.TILE_SIZE);
        int checkCol = (int) Math.round(nextX / Maze.TILE_SIZE);
        return !maze.isWall(checkRow, checkCol);
    }

    public void draw(Graphics g) {
        if (sprite != null) {
            g.drawImage(sprite, (int)x, (int)y, Maze.TILE_SIZE, Maze.TILE_SIZE, null);
        } else {
            g.setColor(Color.YELLOW);
            g.fillOval((int)x, (int)y, Maze.TILE_SIZE, Maze.TILE_SIZE);
        }
    }

    public int getRow() { return row; }
    public int getCol() { return col; }
    public double getX() { return x; }
    public double getY() { return y; }
}

```

GamePanel.java

java

```
import javax.swing.*;
```

```

import java.awt.*;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;

public class GamePanel extends JPanel implements Runnable {
    private Thread gameThread;
    private Maze maze;
    private PacMan pacman;
    private ScoreManager scoreManager;
    private boolean gameOver = false;
    private boolean won = false;

    public GamePanel() {
        maze = new Maze();
        pacman = new PacMan(maze, 1, 1);
        scoreManager = new ScoreManager();

        setBackground(Color.BLACK);
        setPreferredSize(new Dimension(
            Maze.COLS * Maze.TILE_SIZE,
            Maze.ROWS * Maze.TILE_SIZE + 40));
        setFocusable(true);

        addKeyListener(new KeyAdapter() {
            @Override
            public void keyPressed(KeyEvent e) {
                switch(e.getKeyCode()) {
                    case KeyEvent.VK_UP: pacman.setDirection(0, -1); break;
                    case KeyEvent.VK_DOWN: pacman.setDirection(0, 1); break;
                    case KeyEvent.VK_LEFT: pacman.setDirection(-1, 0); break;
                    case KeyEvent.VK_RIGHT: pacman.setDirection(1, 0); break;
                }
            }
        });

        gameThread = new Thread(this);
        gameThread.start();
    }

    @Override
    public void run() {
        while(true) {
            if (!gameOver && !won) {
                pacman.update();

                // Check dot eating
                int eaten = pacman.eatDot();
                if (eaten == 2) scoreManager.addDot();
                if (eaten == 3) scoreManager.addPowerPellet();

                // Check win condition
                if (maze.countDots() == 0) won = true;
            }
            repaint();
            try { Thread.sleep(16); }
            catch (InterruptedException e) { e.printStackTrace(); }
        }
    }
}

```

```

    }

}

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g;
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

    // HUD - Score
    g2.setFont(new Font("Arial", Font.BOLD, 16));
    g2.setColor(Color.WHITE);
    g2.drawString("SCORE", 10, 18);
    g2.setColor(Color.YELLOW);
    g2.drawString(String.valueOf(scoreManager.getScore()), 10, 34);

    // HUD - Lives
    g2.setColor(Color.WHITE);
    g2.drawString("LIVES", 180, 18);
    for (int i = 0; i < scoreManager.getLives(); i++) {
        g2.setColor(Color.YELLOW);
        g2.fillOval(180 + i * 22, 22, 14, 14);
    }

    // Title
    g2.setColor(Color.WHITE);
    g2.setFont(new Font("Arial", Font.BOLD, 16));
    g2.drawString("PAC-MAN", 85, 25);

    // Maze + PacMan
    Graphics2D g2maze = (Graphics2D) g2.create();
    g2maze.translate(0, 40);
    maze.draw(g2maze);
    pacman.draw(g2maze);
    g2maze.dispose();

    // Overlays
    if (won) drawOverlay(g2, "YOUWIN!", Color.YELLOW);
    else if (gameOver) drawOverlay(g2, "GAME OVER", Color.RED);
}

private void drawOverlay(Graphics2D g2, String message, Color color) {
    g2.setColor(new Color(0, 0, 0, 160));
    g2.fillRect(0, 0, getWidth(), getHeight());
    g2.setFont(new Font("Arial", Font.BOLD, 36));
    g2.setColor(color);
    FontMetrics fm = g2.getFontMetrics();
    int x = (getWidth() - fm.stringWidth(message)) / 2;
    int y = getHeight() / 2;
    g2.drawString(message, x, y);
}
}

```

App.java

java

```

public class App {
    public static void main(String[] args) {

```

```

        GameWindow window = new GameWindow();
        window.setVisible(true);
    }
}

```

GameWindow.java

java

```

import javax.swing.JFrame;

public class GameWindow extends JFrame {
    public GameWindow() {
        setTitle("Pac-Man Game");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);
        GamePanel panel = new GamePanel();
        add(panel);
        pack();
        setLocationRelativeTo(null);
    }
}

```

10-WEEK PROGRESS

Week 1 — Setup & Window

✓

Week 2 — Maze Design

✓

Week 3 — Pac-Man Character

✓

Week 4 — Dot Collection & Scoring

✓

Weeks 5–7 — Ghosts + AI

0%

Weeks 8–10 — Polish + Ship

0%

UP NEXT



Ghost Base Class

Create Ghost.java with shared logic for position, speed and state. Load one ghost sprite and implement random maze movement.

Ghost.java · position/speed/state · random movement · sprite loading